WEEK 2 – ENTITY RELATIONSHIP ANALYSIS

E-Commerce Order Management Database System

Document Type: Structural Analysis Map

Author: Lead Database Engineer

Status: Released / Baseline

Target Folder: /Week2/

1. Document Objectives & Structural Scope

This **Entity Relationship Analysis** serves as a deep-dive architecture map companion to the structural schemas defined in the Entity Report. While the previous document prioritized isolated tables and localized column configurations, this document focuses strictly on the **interdependent connectivity vectors, join conditions, mapping constraints, and cascade integrity laws** governing the core components.

Securing transactional safety across complex master networks requires mapping how entities react under extreme conditions, including cascade deletions, concurrent order updates, and foreign constraint validation failures. The target design eliminates potential anomalies to enforce strict referential structures.

2. Cardinality & Participation Matrix

The blueprint matrix below details structural relationships, documenting minimum and maximum participation properties (Optional vs. Mandatory structures) alongside foreign key references.

Parent Entity	Relationship Verb	Child Entity	Cardinality	Participation Profile
Customers	Establishes / Owns	Addresses	1 : N	Parent: OPTIONAL Child: MANDATORY
Customers	Places / Incurs	Orders	1 : N	Parent: OPTIONAL Child: MANDATORY
Categories	Classifies / Houses	Products	1 : N	Parent: OPTIONAL Child: MANDATORY
Orders	Contains Line Items	OrderItems	1 : N	Parent: MANDATORY (1+) Child: MANDATORY
Products	Populates Lines	OrderItems	1 : N	Parent: OPTIONAL Child: MANDATORY
Orders	Triggers Settlement	Payments	1 : 1	Parent: OPTIONAL Child: MANDATORY
Orders	Generates Freight	ShippingDetails	1 : 1	Parent: OPTIONAL Child: MANDATORY

3. Referential Integrity Laws & Cascade Logic

To prevent orphaned database records or missing operational links, foreign keys are governed by strict enforcement settings during delete and update events.

Relational Join Vector	ON DELETE Rule	ON UPDATE Rule	Architectural Justification
`Addresses.CustomerID`	CASCADE	CASCADE	If a customer profile is removed, associated shipping address histories are removed automatically.
`Products.CategoryID`	RESTRICT	CASCADE	Prevents the deletion of active catalog categories if they still contain assigned inventory products.
`Orders.CustomerID`	RESTRICT	CASCADE	Protects core checkout transactions; active customer profiles cannot be deleted if past order entries exist.
`OrderItems.OrderID`	CASCADE	CASCADE	Ensures item details are automatically cleaned up if a master parent order invoice is deleted.
`OrderItems.ProductID`	RESTRICT	CASCADE	Blocks deletion of a product profile if historical sales lines refer to its identifier.
`Payments.OrderID`	RESTRICT	CASCADE	Locks accounting and financial ledger entries against accidental administrative deletion.

4. Structural Boundaries & Business Constraints

4.1. The Resolution of Many-to-Many Relationships

A core challenge in e-commerce workflows is the direct interaction between **Orders** and **Products**. An order contains multiple items, and a product can appear in multiple orders.

To resolve this structural Many-to-Many dependency, an associative entity named **OrderItems** is established. Beyond mapping foreign keys, this entity records point-in-time metrics: the exact sales price (`UnitPrice`) and applied discount levels (`Discount`) at the time of purchase. This guarantees that future price updates in the master inventory table do not retroactively alter historical financial metrics.

4.2. Double-Vector Foreign Key Routing (Addresses Matrix)

The relationship layout requires careful routing between the **Orders** and **Addresses** schemas. Every order needs two separate geographic attributes:

- `ShippingAddressID` points to the target package delivery node.
- `BillingAddressID` tracks the point-of-origin context for payment authorization checks.

This creates a double foreign key vector pointing to the same lookup table (`Addresses`). This design pattern keeps all customer locations in a single optimized table while maintaining independent tracking for shipping and billing contexts.

4.3. One-to-One Isolation Strategies

The system uses strict One-to-One constraints for **Payments** and **ShippingDetails** relative to their parent **Orders**.

By adding a `UNIQUE` index constraint over the `OrderID` attribute in both tables, the database engine guarantees that an invoice cannot have multiple payment records or duplicate shipping tracking manifests. This pattern minimizes duplicate operations and simplifies accounting lookups.

5. Relational Join Vectors & Logical Flow

The summary below outlines the relational pathways used by developers and application query engines to execute multi-table joins:

```
[CUSTOMERS] (1) ──────────────────────────────── (N) [ADDRESSES]
             \ (1)
```

```
               \
               (N) [ORDERS] (1) ─────────────────────── (1) [PAYMENTS] (Unique Unique)
                    | (1)
                    | ─────────────────────────────── (1) [SHIPPING_DETAILS] (Unique)
                    | (1)
                    \─── (N) [ORDER_ITEMS] (N) ────────────── (1) [PRODUCTS]
                                                                               |
                                                                               | (N)
                                                                               | (1)
                                                                        [CATEGORIES]
```

6. Operational Rules Verification Checklist

- **Constraint Rule Validation:** Every foreign key is indexed to speed up relational joins and reduce search bottlenecks under high transactional volume.
- **Data Integrity Architecture:** Optional relationships are carefully handled at the application layer to prevent database null anomalies.
- **Normalization Level Statement:** The relational structure satisfies Third Normal Form (3NF) requirements. Functional dependencies map directly to the primary key, removing data redundancy.

Prepared & Mapped By: Relational Database Architect
Data Infrastructure Division

Approved & Verified By: Engineering Review Committee
E-Commerce Architecture Team